

Mobile React Native Developer Technical Interview Q&A

1. What is React Native and how does it differ from React?

React Native is a framework for building mobile applications using React and JavaScript. While React targets web browsers and uses HTML/CSS, React Native targets mobile platforms (iOS and Android) and uses native components. React Native compiles to native code, providing near-native performance. The main difference is that React Native uses native mobile components (View, Text, Image) instead of web elements (div, span, img), and it uses StyleSheet API instead of CSS. This allows code sharing between platforms while maintaining native look and feel.

2. Explain the React Native architecture and how it works.

React Native uses a bridge architecture where JavaScript code runs in a separate thread and communicates with native modules through an asynchronous bridge. The JavaScript thread handles business logic and React components, while native threads handle UI rendering and platform-specific APIs. When JavaScript needs to update the UI, it sends serialized messages across the bridge to native modules. This architecture allows React Native to leverage native performance while using JavaScript for development. Newer architectures (Fabric and TurboModules) are reducing bridge overhead for better performance.

3. What is the difference between React Native and native mobile development?

Native development uses platform-specific languages (Swift/Objective-C for iOS, Kotlin/Java for Android) and provides the best performance and full access to platform features. React Native uses JavaScript and allows code sharing between platforms, reducing development time and maintenance. Native apps typically have better performance and access to the latest platform features immediately, while React Native offers faster development, easier maintenance, and a single codebase. The choice depends on performance requirements, team expertise, and development timeline.

4. Explain React Native's component lifecycle and hooks.

React Native components follow the same lifecycle as React web components. Class components use lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. Functional components use hooks: `useState` for state, `useEffect` for side effects (equivalent to `componentDidMount/Update`), `useCallback` and `useMemo` for optimization, and custom hooks for reusable logic. `useEffect` can handle mounting, updating, and cleanup by returning a cleanup function. Understanding lifecycle is crucial for managing resources, subscriptions, and API calls properly.

5. How do you handle navigation in React Native?

Navigation in React Native is typically handled by React Navigation, the most popular library. It provides stack navigation (screen stacking), tab navigation (bottom/top tabs), drawer navigation (side menu), and nested navigators. React Navigation uses a declarative API where you define your navigation structure, and it handles transitions, deep linking, and state management. Alternative libraries include React Native Navigation (more native, better performance) and React Router Native. Navigation state is managed centrally, allowing programmatic navigation and passing parameters between screens.

6. What is the difference between state and props in React Native?

Props are read-only data passed from parent to child components, similar to function parameters. They're immutable and cannot be changed by the child. State is mutable data managed within a component using `useState` hook. When state changes, React Native re-renders the component. Props flow down the component tree, while state is local unless lifted to a common ancestor. For global state, you'd use Context API, Redux, or other state management solutions. Understanding this distinction is fundamental to React Native development.

7. How do you handle API calls and data fetching in React Native?

API calls in React Native use `fetch()` or libraries like `axios`, similar to web React. Data fetching typically happens in `useEffect` hooks with proper dependency arrays. You manage loading states, errors, and data using `useState`. For more complex scenarios, libraries like `React Query` or `SWR` provide caching, background refetching, and better state management. It's important to handle network errors, implement retry logic, and show appropriate loading indicators. For offline support, you might use libraries like `React Query` with persistence or implement custom caching strategies.

8. Explain the StyleSheet API in React Native.

React Native uses `StyleSheet.create()` instead of CSS files. Styles are JavaScript objects with camelCase properties (`backgroundColor` instead of `background-color`). `StyleSheet` provides performance optimizations by creating style objects once and reusing them, and it validates styles at creation time. You can combine styles using arrays, and styles cascade but don't support all CSS features. Flexbox is the primary layout system, and you can use absolute positioning. Platform-specific styles can be applied using `Platform.OS` checks or `Platform.select()`.

9. What are React Native's performance optimization techniques?

Performance optimization includes: using `React.memo` to prevent unnecessary re-renders, implementing `FlatList` or `SectionList` for long lists (instead of `ScrollView` with `map`), using `useMemo` and `useCallback` for expensive computations, optimizing images (resizing, using appropriate formats), reducing bridge communication, using native driver for animations, implementing code splitting, and avoiding inline functions and object creation in render. The Hermes JavaScript engine (enabled by default in newer versions) also improves startup time and memory usage.

10. How do you handle images in React Native?

React Native provides the Image component for displaying images. You can load images from local assets (`require('./image.png')`), network URLs, or base64 data. For network images, you should handle loading states and errors. The Image component supports `resizeMode` (`cover`, `contain`, `stretch`, `repeat`, `center`) for different display needs. For better performance, you can use libraries like `react-native-fast-image` for caching and optimization. It's important to optimize image sizes and use appropriate formats (WebP for Android, HEIC for iOS when possible).

11. Explain state management in React Native applications.

State management options include: local state (`useState` for component-specific data), Context API (for sharing data across components without prop drilling), Redux/Redux Toolkit (for complex global state), Zustand or Jotai (lighter alternatives to Redux), and React Query (for server state). The choice depends on app complexity - simple apps might only need `useState` and Context, while complex apps benefit from Redux or similar solutions. React Query is excellent for server state (API data) as it handles caching, synchronization, and background updates automatically.

12. What are React Native bridges and native modules?

Bridges enable communication between JavaScript and native code. When React Native doesn't have a built-in API for something, you create native modules - code written in native languages (Swift/Objective-C for iOS, Kotlin/Java for Android) that can be called from JavaScript. Native modules are useful for accessing platform-specific features like camera, Bluetooth, biometrics, or third-party SDKs. You define the interface in native code, create a bridge, and expose it to JavaScript. Libraries like `react-native-create-bridge` simplify this process.

13. How do you handle offline functionality in React Native?

Offline functionality can be implemented using: AsyncStorage or MMKV for local data persistence, NetInfo to detect network status, React Query with persistence for caching API responses, Redux Persist for state persistence, and implementing sync logic to upload changes when online. You should cache critical data, queue actions when offline, and sync when connectivity returns. Libraries like WatermelonDB or Realm provide local databases for complex offline scenarios. Proper offline handling significantly improves user experience.

14. Explain the difference between FlatList and ScrollView.

ScrollView renders all children immediately, which is fine for small lists but causes performance issues with many items. FlatList is virtualized - it only renders visible items and recycles views as you scroll, making it efficient for long lists. FlatList provides built-in features like pull-to-refresh, loading indicators, empty states, and section headers. Use ScrollView for small, static content, and FlatList for dynamic lists of data. SectionList extends FlatList for grouped data with section headers.

15. How do you handle forms and user input in React Native?

Forms in React Native use TextInput components controlled by state. You manage input values with useState and handle changes with onChangeText. For complex forms, libraries like Formik or React Hook Form provide validation, error handling, and form state management. TextInput supports various props like placeholder, keyboardType, secureTextEntry, multiline, and autoCapitalize. You should handle different keyboard types, implement proper validation, show error messages, and manage focus between inputs. For better UX, consider using libraries like react-native-paper or native-base that provide enhanced form components.

16. What is the Hermes JavaScript engine and why is it important?

Hermes is a JavaScript engine optimized for React Native, developed by Facebook. It improves app performance by: reducing memory usage, decreasing app startup time, and providing better time-to-interactive metrics. Hermes compiles JavaScript to bytecode ahead of time,

which is faster to parse and execute. It's enabled by default in React Native 0.70+ and provides significant performance improvements, especially on Android. Hermes makes React Native apps feel more native and responsive.

17. How do you handle push notifications in React Native?

Push notifications require: setting up Firebase Cloud Messaging (FCM) for Android and Apple Push Notification service (APNs) for iOS, configuring native permissions, and using libraries like `react-native-push-notification` or `@react-native-firebase/messaging`. You need to request notification permissions, handle token registration, set up notification handlers for foreground/background/quit states, and implement deep linking if notifications should open specific screens. Testing requires physical devices or proper simulator configuration, as push notifications don't work in standard simulators.

18. Explain code pushing and over-the-air (OTA) updates in React Native.

Code pushing allows updating JavaScript code without going through app store review, using services like CodePush (Microsoft) or EAS Update (Expo). This enables quick bug fixes and feature rollouts. However, you cannot update native code - only JavaScript bundles. OTA updates are useful for: fixing critical bugs quickly, A/B testing, gradual feature rollouts, and updating business logic. Native code changes still require app store updates. CodePush integrates with CI/CD pipelines and supports staged rollouts and rollbacks.

19. How do you handle deep linking in React Native?

Deep linking allows opening specific app screens via URLs. Implementation involves: configuring URL schemes (`myapp://`) or universal links (`https://myapp.com`), handling incoming links in the app, parsing URL parameters, and navigating to appropriate screens. Libraries like `react-native-deep-linking` or React Navigation's linking configuration simplify this. You need to handle cases where the app is closed, in background, or already open. Deep linking is essential for: email

verification, password resets, sharing content, and marketing campaigns directing users to specific app sections.

20. What are React Native's platform-specific code patterns?

Platform-specific code can be implemented using: Platform.OS to check the platform ('ios' or 'android'), Platform.select() for platform-specific values, separate files with .ios.js and .android.js extensions, and conditional rendering. This allows you to provide different implementations, styles, or components for each platform while sharing most of your codebase. Platform-specific code is necessary when: using platform-exclusive features, implementing different UI/UX per platform guidelines, or handling platform-specific APIs.

21. How do you handle animations in React Native?

React Native provides Animated API for JavaScript-driven animations and react-native-reanimated for better performance with worklets running on the UI thread. The Animated API is simpler but runs on the JS thread, while Reanimated 2 runs animations on the UI thread for 60fps performance. For layout animations, you can use LayoutAnimation. Common animation use cases include: screen transitions, gesture-based animations, loading indicators, and micro-interactions. Reanimated is recommended for complex animations and when performance is critical.

22. Explain error handling and crash reporting in React Native.

Error handling involves: using try-catch blocks for synchronous code, handling promise rejections, implementing error boundaries (though limited in React Native), and using global error handlers. For crash reporting, services like Sentry, Bugsnag, or Crashlytics capture crashes, provide stack traces, and help debug production issues. You should log errors appropriately, provide user-friendly error messages, and implement retry logic for network failures. Proper error handling and reporting are crucial for maintaining app stability and quickly fixing issues.

23. How do you test React Native applications?

Testing strategies include: unit tests with Jest for business logic, component tests with React Native Testing Library, integration tests for feature flows, and E2E tests with Detox or Appium. Jest is configured by default and can test JavaScript code. React Native Testing Library provides utilities for testing components. Detox enables E2E testing on simulators/emulators. Testing native modules requires additional setup. You should test critical user flows, edge cases, and ensure tests run in CI/CD pipelines. Good test coverage helps catch regressions and enables confident refactoring.

24. What is the difference between Expo and React Native CLI?

Expo is a framework and platform built around React Native that simplifies development with: managed workflow (no native code access initially), built-in APIs and services, over-the-air updates, easier builds and deployments, and extensive documentation. React Native CLI provides full control over native code from the start but requires more setup and knowledge of native development. Expo is great for rapid prototyping and apps that don't need custom native modules, while CLI is better when you need full native control or specific native libraries not supported by Expo.

25. How do you handle app state persistence in React Native?

State persistence can be implemented using: AsyncStorage for simple key-value storage, MMKV for better performance, Redux Persist for Redux state, React Query with persistence for server state, or databases like WatermelonDB or Realm for complex data. You should persist: user authentication tokens, user preferences, cached data, and offline queues. It's important to handle migration when data structure changes and secure sensitive data (using Keychain on iOS, Keystore on Android). State persistence ensures users don't lose data and provides better offline experience.

26. Explain React Native's security best practices.

Security practices include: storing sensitive data in secure storage (Keychain/Keystore), using certificate pinning for API calls, obfuscating JavaScript code, validating and sanitizing all inputs, using HTTPS for all network requests, implementing proper authentication and authorization, avoiding storing sensitive data in AsyncStorage, and keeping dependencies updated. For highly sensitive apps, you might need additional measures like root/jailbreak detection, anti-tampering, and runtime application self-protection (RASP). Security is especially important for financial, healthcare, or apps handling personal data.

27. How do you handle app performance monitoring?

Performance monitoring involves: tracking app startup time, measuring screen render times, monitoring JavaScript thread performance, tracking native module performance, using tools like Flipper or React Native Debugger, implementing custom performance markers, and using services like Firebase Performance Monitoring or New Relic. You should identify bottlenecks, optimize slow screens, reduce bundle size, and monitor memory usage. Performance monitoring helps identify issues before users experience them and guides optimization efforts. Regular profiling is essential for maintaining good app performance.

28. What are React Native's memory management considerations?

Memory management involves: properly cleaning up subscriptions and listeners in `useEffect` cleanup, avoiding memory leaks from closures, managing image caching appropriately, releasing native module resources, avoiding circular references, and monitoring memory usage. React Native apps can have memory issues from: large images not being released, listeners not being removed, large arrays/objects in state, and native module leaks. Tools like Flipper's memory profiler help identify leaks. Proper cleanup in `useEffect` return functions is crucial for preventing memory leaks.

29. How do you handle app updates and versioning?

App updates require: semantic versioning (major.minor.patch), maintaining changelogs, implementing update prompts for critical updates, using code pushing for JavaScript updates when possible, and planning app store releases. You should: test updates thoroughly, support backward compatibility when possible, communicate changes to users, and have rollback plans. For native updates, you need to go through app store review, which takes time. Code pushing allows faster JavaScript updates. Version management helps track what's deployed and enables feature flags or gradual rollouts.

30. Explain the React Native development workflow and tools.

Development workflow typically involves: setting up development environment (Node.js, React Native CLI or Expo), using Metro bundler for JavaScript bundling, running on simulators/emulators or physical devices, using hot reloading for fast iteration, debugging with React Native Debugger or Flipper, using ESLint and Prettier for code quality, implementing Git workflows, and setting up CI/CD for automated testing and building. Essential tools include: VS Code with React Native extensions, Flipper for debugging, Fastlane for deployment automation, and app store developer accounts. A good workflow significantly improves development speed and code quality.